

USE CASE CONCEPT

Agentic onboarding for containerized applications

Prompt-driven platform provisioning that turns “I need to onboard a 25-service app” into governed infrastructure, networking, ingress, policies, scaling, and validation.

AI + Platform Engineering

Self-Service

Guarded Automation

The problem: onboarding does not scale with service complexity

A 25-service application multiplies manual setup, approvals, and configuration decisions.

CURRENT FRICTION

Forms require users to already know infra needs

Developers must translate app intent into platform language.

Setup spans multiple domains

Infra, networking, ingress, policies, identity, observability, and scaling.

Manual handoffs create delay

Missing fields, policy exceptions, and environment-specific rules slow delivery.

Support teams get repetitive tickets

The same onboarding questions and troubleshooting paths repeat.

25

application
services

5+

domains to
coordinate

Nx

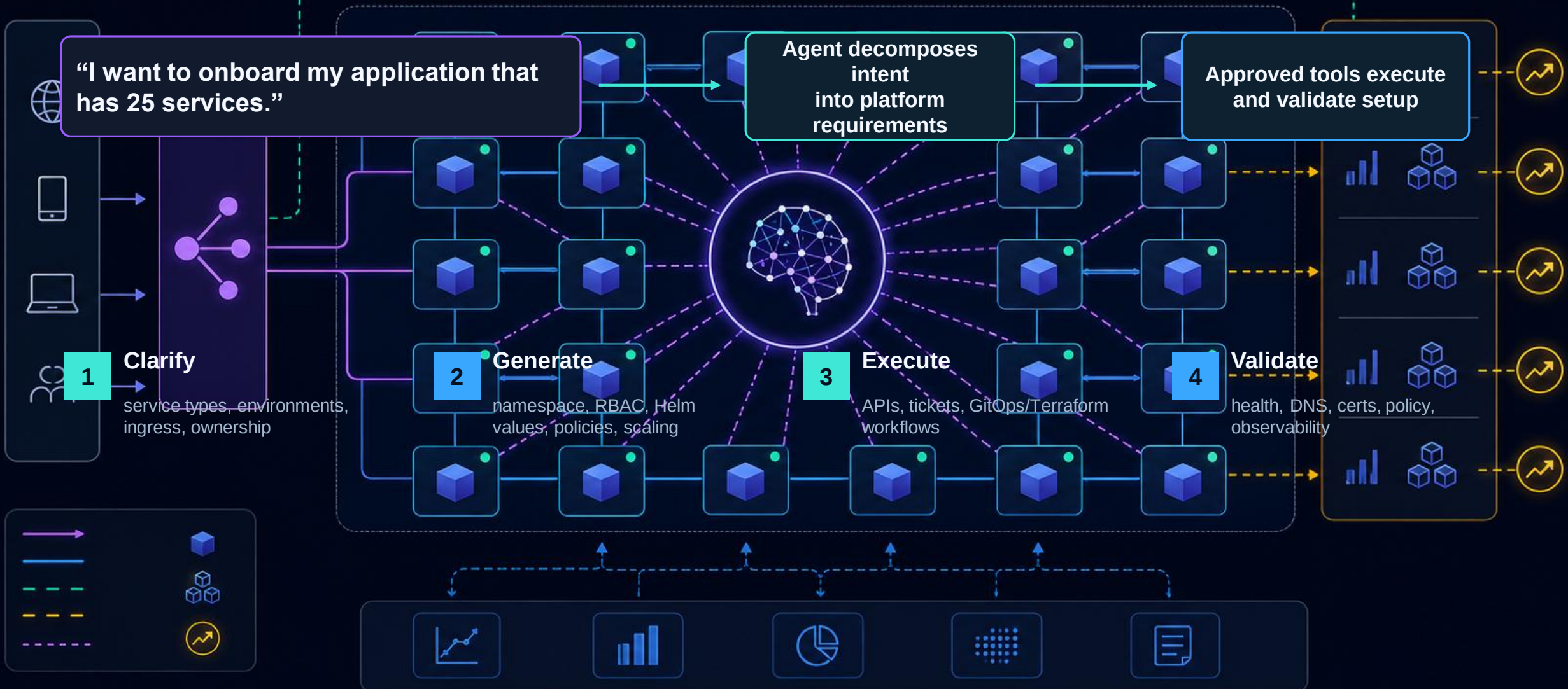
policy +
ingress
decisions

**The user's real goal is not "fill out the form."
It is: "make my app ready to run safely."**

Agentic onboarding starts from intent, not infrastructure jargon.

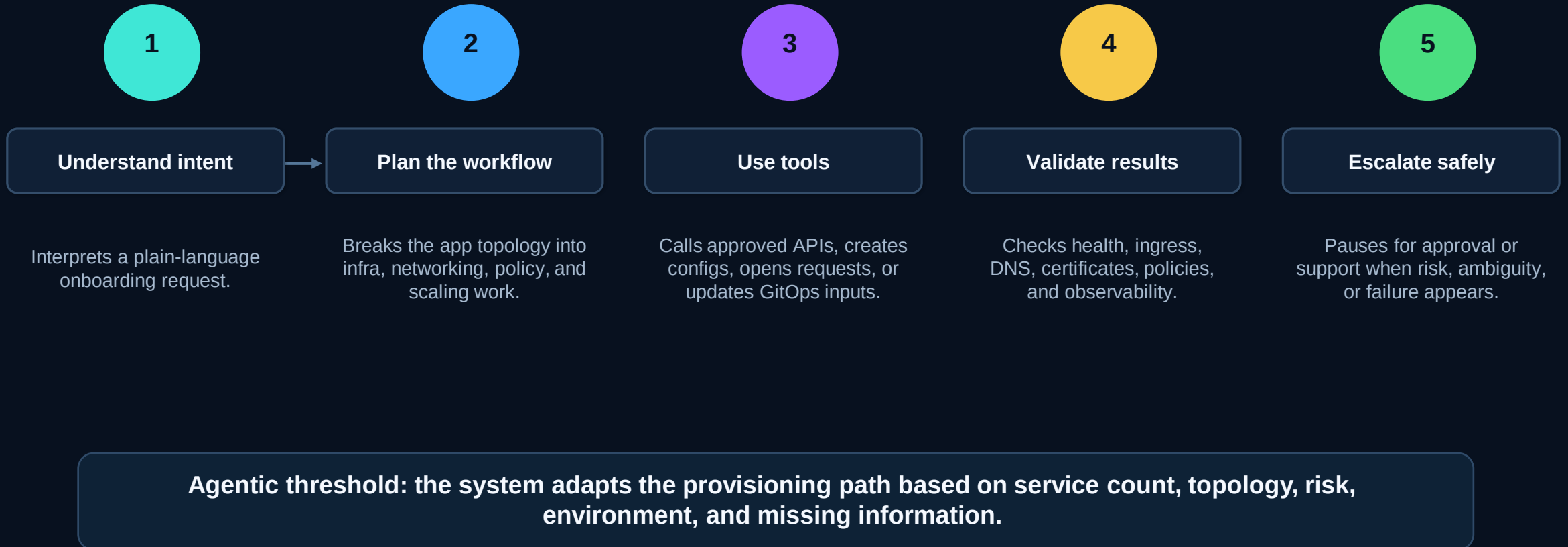
The vision: prompt → plan → provision → validate

The agent converts a natural-language onboarding request into a governed provisioning workflow.



Why this is agentic — not just automation

The agent does more than run a predefined script. It reasons through the goal, selects a path, uses tools, and checks the outcome.



Example user experience: from one prompt to a complete onboarding plan

The user starts with intent, the agent progressively structures the request.

CONVERSATION

User: I want to onboard my application that has 25 services.

Agent: Which services need external ingress, internal access, or worker-only processing?

User: 3 public APIs, 10 internal APIs, 12 workers across dev/test/prod.

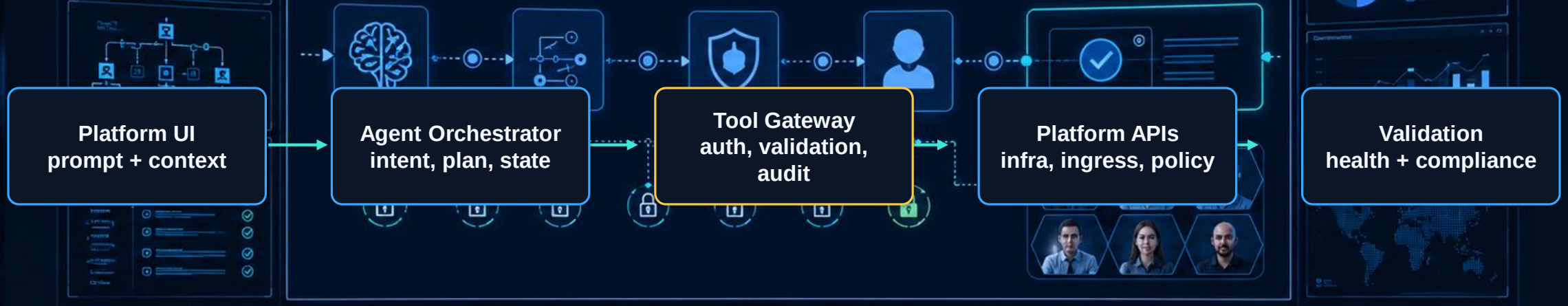
Agent: I recommend environment-specific namespaces, 3 ingress routes, internal ClusterIP services, HPA for APIs, KEDA for workers, and default-deny network policies.

GENERATED PLAN

- ✓ 3 namespaces per environment pattern
- ✓ 25 deployments + service definitions
- ✓ 3 ingress routes + certificate workflow
- ✓ RBAC, service accounts, secrets integration
- ✓ Network policies + allowed service paths
- ✓ HPA/KEDA scaling + resource quotas
- ✓ Logs, metrics, traces, alerts, dashboard
- ✓ Approval gates for prod/external exposure

Reference architecture: governed tool-using agent

Keep the model powerful, but put execution behind controlled platform services.



TOOL EXAMPLES

Create namespace

Generate Helm values

Apply policies

Configure ingress

Create ticket

Check deployment

Register DNS

Create dashboard

Design principle: the agent does not directly mutate infrastructure. It requests actions through allowlisted, audited, policy-aware tools.

25-service topology: how the agent scales the onboarding pattern

The agent classifies the service mix, then generates the right platform resources for each class.

3

Public APIs

external ingress + certs + HPA

10

Internal APIs

ClusterIP + service mesh + policies

12

Workers

no ingress + queue/event scaling

Namespaces

dev / test / prod boundaries

Deployments

25 workload specs

Services

public, internal, worker patterns

Ingress

routes, DNS, cert workflows

Network policy

default deny + explicit allow

Identity

service accounts + RBAC

Scaling

HPA + KEDA + quotas

Observability

logs, metrics, traces, alerts

Validation

health, policy, reachability

Governance model: agency with boundaries

The system can act, but only through approved controls, approval gates, and audit trails.

Identity-aware

SSO, user context, RBAC, ownership checks

Policy-aware

OPA/Kyverno-style rules before changes

Approval-aware

pause for prod, external ingress, exceptions

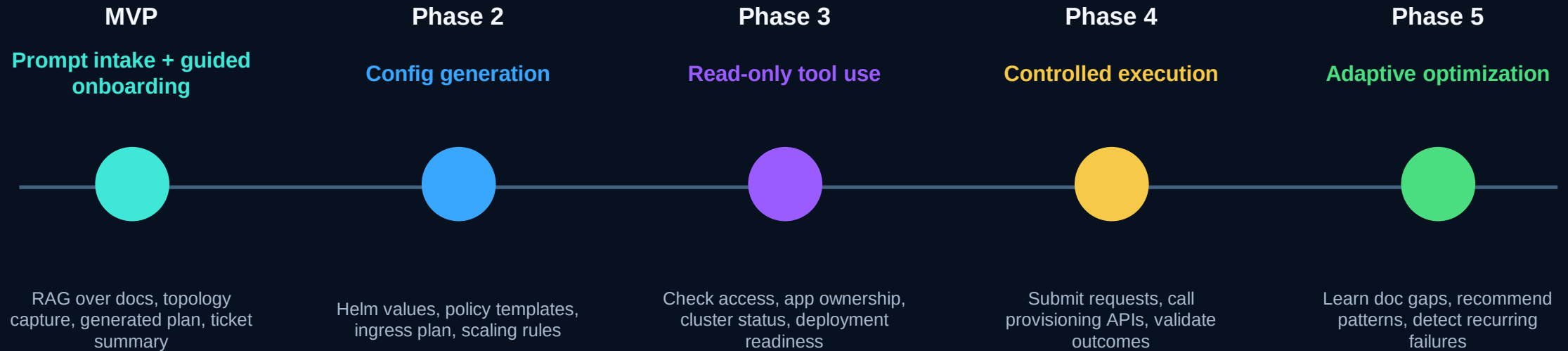
Audit-aware

record prompts, decisions, tool calls, outcomes

Human-in-the-loop is not a limitation — it is what makes enterprise agentic automation trustworthy.

Roadmap: start with guided onboarding, then add controlled execution

Build trust through constrained workflows before expanding agent autonomy.



Recommended launch posture: Level 3–4 agentic automation first — tool-using, policy-aware, approval-gated.

Success measures: prove value across product, operations, and risk

The agent should improve developer experience without weakening platform control.

Product outcomes

- Faster onboarding completion
- Higher self-service success
- Lower abandonment
- Improved user satisfaction

Operational outcomes

- Fewer repetitive tickets
- Shorter support handoffs
- Higher tool success rate
- Lower provisioning rework

Risk outcomes

- Policy-compliant changes
- Approval capture
- Audit-ready tool calls
- Reduced misconfiguration

End state: developers describe intent; the platform produces a safe, validated, production-ready onboarding path.